



GGoBOOK

웹툰, 웹소설 통합 플랫폼

함께 읽고 함께 쓰는 이야기

1조 (김승경, 김재민, 강한수, 임성훈)

2026. 05. 11

github.com/ggobook-project

목차

Table of Contents

01 프로젝트 배경 및 개요

03 기대 효과

05 구성도

07 주요 기능 소개

09 자체 평가 및 의견

02 팀 구성 및 역할

04 기술 스택

06 프로젝트 시연

08 SWOT 분석

프로젝트 배경 및 개요

Background & Overview

배경

 국내 웹툰 시장 약 2조 원 돌파
매년 두 자릿수 성장

 웹툰·웹소설 분리 운영
통합된 사용자 경험 부재

 AI를 접목한 플랫폼 부재
기술 차별화 기회 존재

 신인 작가 진입 장벽 높음
누구나 등록 가능한 환경 필요

개요

GGoBOOK은 웹툰과 웹소설을 **하나의 플랫폼**에서 즐기는 통합 콘텐츠 플랫폼입니다.

독자는 작품을 읽고, 작가는 직접 등록·연재할 수 있으며,
포인트 결제와 다양한 AI 기능을 지원합니다.

팀 구성 및 역할

Team & Roles



김승경

팀장 · Backend / AI

결제 · 동시성 처리
작품/회차 CRUD
검색 · 태그 · AWS S3
평행우주 외전
꼬북이 챗봇



김재민

Frontend · TTS API

전체 UI / UX
작품·관리자 페이지
마이페이지 · 결제
릴레이 / 로그인
TYPECAST TTS 연동



강한수

Backend / AI

정지·신고·관리자
회원·공지·알림
소셜 요약 AI
AI 검수
릴레이 소셜 백엔드



임성훈

Backend

일반/소셜 로그인
댓글·답글
평점·찜·좋아요
마이페이지 API

협업 방식

GitHub Flow

main / develop / feature/기능명

SourceTree

Git GUI · 브랜치 시각화 협업

API 명세 공유

프론트·백 동기화된 협업

기대 효과

Technical & Operational



기술적 기대 효과

01 Spring Security + JWT + OAuth2

인증/인가 실무 구현 역량

02 포트원 결제 API 연동

Race Condition 동시성 처리 포함

03 FastAPI + Gemini API

LLM 서비스 + 프롬프트 엔지니어링

04 Typecast TTS API

한국어 특화 음성 합성

05 AWS S3 클라우드

파일 업로드 인프라 활용 역량



운영적 기대 효과

01 통합 플랫폼

작가-독자 연결로 운영 효율화

02 검수 프로세스 체계화

DRAFT → PENDING → APPROVED → PUBLISHED

03 관리자 자동화

신고 · 정지 · 블라인드 운영

04 사용자 행동 데이터

Reading / RecentView 기반 개인화

05 AI 검수 + 랭킹 자동화

운영 인력 절감 + 인기 작품 노출

기대 효과

Social Impact · Differentiation · Revenue

사회적 기대 효과

- 신인 작가 창작 진입 장벽 낮춤 (누구나 등록)
- 릴레이 소설로 협업 창작 문화 형성
- TTS로 시각 장애인 등 접근성 향상
- 평행우주 외전 — 능동적 콘텐츠 소비 문화

매출 / 수익 모델

 유료 회차 구매
회차당 200P

 광고 수익
배너 / 스폰서

 작가 수익 분배
콘텐츠 경쟁력 강화

 월정액 구독
안정적 수익 확보

시장 차별화

구분	GGoBOOK	기존 플랫폼
콘텐츠	웹툰 + 웹소설 통합	웹툰 또는 웹소설 분리
AI 기능	챗봇 · TTS · 외전 · 검수	거의 없음
창작 참여	누구나 작품 등록	계약 작가 위주
릴레이	협업 창작 기능 제공	없음

기술 스택

Tech Stack

Frontend

React

SPA + React Router DOM + Context API

Vite

빌드 도구

Axios

Spring Boot / FastAPI 분리 인스턴스

CSS Modules

컴포넌트 단위 스타일링

UI Libraries

Swiper, React DatePicker

Backend

Spring Boot 3

REST API 서버

Spring Security + JWT

인증 / 인가

OAuth2

카카오 · 구글 · 네이버

JPA / Hibernate

ORM · MySQL

PortOne V1

결제 API

AI / LLM

FastAPI

AI 전용 서버 (8000)

Google Gemini API

챗봇 · 외전 · 검수

TYPECAST API

한국어 특화 TTS

Multi-voice TTS

화자 구분 자동 변환

프롬프트 엔지니어링

도메인 특화 응답

Infrastructure

AWS EC2 (Ubuntu)

t3.medium · 서울 리전

AWS RDS MySQL

8.0.x · ggobook-db

AWS S3

이미지 / 파일 저장

Docker / Compose

3개 컨테이너 통합 관리

GitHub

레포 3개 (BE/FE/LLM)

시스템 구성도

System Architecture



ERD

Entity-Relationship Diagram

사용자

user

14 cols

결제 / 포인트

payment

10 cols

point

6 cols

wallet

7 cols

운영 / 관리

report

9 cols

member_suspend

7 cols

notice

7 cols

notification

8 cols

콘텐츠

content

16 cols

episode

11 cols

content_tag

4 cols

comic_toon

4 cols

novel

3 cols

TTS

tts_voice

8 cols

tts_episode_setting

6 cols

tts_chunk

7 cols

multi_voice_chunk

9 cols

참여 / 평가

comment

9 cols

comment_reaction

5 cols

reply

5 cols

reply_reaction

5 cols

likes

4 cols

episode_like

5 cols

rating

6 cols

owned_content

5 cols

reading

5 cols

recent_view

5 cols

릴레이 협업

relay_topic

5 cols

relay_novel

7 cols

relay_entry

6 cols

admin_relay_topic

5 cols

relay_guideline

5 cols

클래스 다이어그램

Class Diagram · Spring Boot 3-Tier Architecture

Auth / User 인증·회원

@RestController · 3개

Auth, SocialLogin, FindAccount

@Service · 3개

Auth, SocialLogin, FindAccount

@Repository · 1개

User

Relay 릴레이 협업

@RestController · 2개

RelayNovel, RelayEntry

@Service · 2개

RelayNovel, RelayEntry

@Repository · 5개

RelayNovel, RelayEntry +3

Content 작품·회차

@RestController · 3개

Content, Episode, Search

@Service · 3개

Content, Episode, Search

@Repository · 3개

Content, Episode, EpisodelImage

TTS AI 음성합성

@RestController · 1개

TTS

@Service · 1개

TTS

@Repository · 4개

TTSVoice, EpisodeSetting +2

Engagement 댓글·평가·소장

@RestController · 8개

Comment, Reply, CommentLike +5

@Service · 7개

Comment, Reply, CommentLike +4

@Repository · 9개

Comment, Reply, Reaction +6

Notice / Report 공지·신고·알림

@RestController · 4개

Notice, Report, Ranking +1

@Service · 4개

Notice, Report, Ranking +1

@Repository · 4개

Notice, Report, Ranking +1

Payment / Point 결제·포인트

@RestController · 3개

Payment, Point, Wallet

@Service · 3개

Payment, Point, Wallet

@Repository · 3개

Payment, Point, Wallet

Admin 관리자

@RestController · 7개

AdminInspection, AdminMember +5

@Service · 7개

AdminInspection, AdminMember +5

@Repository · 1개

MemberSuspend

Controller 31 · Service 30 · Repository 30 · Util 6 · Entity 32 · 8개 도메인

사이트 흐름도

Site Flow Map

 메인 페이지



인증

로그인 / 회원가입

- 일반 로그인
- 소셜 로그인 (카카오/구글/네이버)
- 아이디·비밀번호 찾기



콘텐츠

웹툰 / 웹소설 탐색

- 작품 목록 → 상세 → 회차 구매
- 웹소설 뷰어 (TTS · 평행우주)
- 검색 · 랭킹 · 태그



창작

작가 / 릴레이

- 작품 등록 / 회차 등록
- 릴레이 소설 (등록 · 이어쓰기)
- 공지사항



마이페이지

개인 활동 관리

- 내 정보 · 포인트 · 충전 내역
- 소장 / 최근 본 / 좋아요 작품
- 내 댓글 · 내 릴레이 소설



작가 페이지

작품 · 회차 관리

- 작품 관리 / 등록 / 수정
- 회차 관리 / 등록 / 수정
- 예약 업로드



관리자

운영 / 검수

- 검수 (PENDING → 승인/반려)
- 회원·신고 처리 · 블라인드
- 공지·릴레이·TTS 관리

프로젝트 시연

Live Demo Walkthrough

01

메인 페이지

랜딩 화면 · 파도 배경

고북이 AI 챗봇

02

회원가입 / 로그인

카카오 · 구글 · 네이버

OAuth2 소셜 로그인

03

마이페이지 · 작품 등록

원고 업로드 · 작가 기능

AI 따옴표 변환 · 자동 요약

04

포인트 충전

충전 → 유료 회차 구매

포인트 결제 시스템

05

웹소설 · 웹툰 뷰어

본문 / 이미지 감상

TTS · 평행우주 외전

06

릴레이 소설

주제 등록 → 이어쓰기

동시 작성 락 처리

07

관리자 페이지

운영 · 검수 · 회원 관리

블라인드 · 검수 · TTS 설정

주요 기능 _ 꼬북이 AI 챗봇

Conversational AI with Real-time DB Context

← 시연 메뉴

📋 챗봇 응답 흐름



⚡ 핵심 설계

🗄️ 실시간 DB 컨텍스트 주입

전체 작품 + JWT 기반 읽은 작품 모두 조회 → Gemini에 컨텍스트 주입

```
# 의도 감지 → DB 컨텍스트 주입
if is_recommend_question(message):
    contents = get_contents()
    read = get_read_contents(token) # JWT
# Gemini 2.5 Flash Lite 호출
response = client.generate_content(...)
```

🔍 키워드 기반 의도 감지

"추천" · "작품" · "FAQ" 등 키워드로 응답 분기

💬 FAQ · 작품 추천 · 대화 초기화

자주 묻는 질문 패널 토글 + 에러 재시도 + 로딩 애니메이션

주요 기능 _ OAuth2 소셜 로그인

Kakao · Google · Naver Authentication

← 시연 메뉴

로그인 흐름

1

사용자 소셜 로그인 버튼 클릭
(카카오 / 구글 / 네이버)

2

OAuth Provider 인증 페이지
(소셜 계정으로 로그인)

3

Authorization Code 발급
(콜백 URL로 리다이렉트)

4

Access Token + 사용자 정보 조회
(Provider API 호출)

5

JWT 발급 + 회원 가입/로그인 처리
(Refresh Token 저장)

핵심 설계



Spring Security OAuth2 Client

OAuth2Attribute 팩토리로 3개 Provider 응답 구조 통일

```
// provider = "kakao"|"google"|"naver"
OAuth2Attribute attr = OAuth2Attribute.of(
    provider, attributeKey, attributes);
// 세 제공자 응답 구조 통일화
String email = attr.getEmail();
String name = attr.getName();
```



JWT 토큰 발급

Access Token (단기) + Refresh Token (장기) 분리 발급



최초 로그인 시 자동 회원가입

Provider ID 매핑 → 별도 가입 절차 없이 바로 이용

주요 기능 _ AI 따옴표 자동 변환

Auto Quote Detection for Multi-Voice Mapping

← 시연 메뉴

변환 흐름

1

작가 원고 등록
(웹소설 일반 텍스트)

2

AI 따옴표 변환 요청
(Gemini API 호출)

3

대사 / 서술 자동 분리
(Gemini가 따옴표 자동 삽입)

4

멀티보이스 포맷 저장
(MultiVoiceChunk 테이블)

5

TTS 변환 즉시 가능
(보이스 매핑 완료 상태)

핵심 설계

AI + Fallback 2단계 처리

Gemini로 따옴표 감지 → 실패 시 정규식 자동 폴백

1안: Gemini로 따옴표 감지

try:

```
formatted = gemini.generate(prompt,  
                             model="gemini-2.0-flash")
```

except: # 2안: 정규식 폴백

```
formatted = _regex_format(text)
```

규칙 기반 화자 매핑

홀수 번째 대사 → 화자1, 짝수 → 화자2 (순서 기반)

작업 시간 단축

수동 따옴표 작업 부담 제거 → 등록 즉시 TTS 사용 가능

주요 기능 _ 포인트 결제 시스템

Payment System with Concurrency Control

← 시연 메뉴

결제 흐름

1

포인트 충전 요청 → preparePayment
(merchantUid 생성, PENDING 저장)

2

포트원 결제창 (프론트)

3

verifyPayment → 포트원 서버 검증

4

COMPLETE 저장 + Wallet.balance 증가
+ Point 내역 저장

5

유료 회차 클릭 → 200P 차감
→ OwnedContent 저장

핵심 설계



Race Condition 방어

낙관적 락(Wallet) + 비관적 락(Payment) — 책임 분리

```
// Wallet.java - 낙관적 락
@Version
private Long version;
// PaymentRepository.java - 비관적 락
@Lock(LockModeType.PESSIMISTIC_WRITE)
Payment findByMerchantUidWithLock(uid);
```



Webhook 동기화

포트원 → 서버 자동 상태 동기화 (permitAll 엔드포인트)



취소 / 환불

포트원 CancelData 연동 → Wallet 잔액 차감 → Point DEDUCT 저장

주요 기능 _ TTS (Text-To-Speech)

AI Voice Narration · Single & Multi-Voice

← 시연 메뉴

TTS 처리 흐름

1

웹소설 본문 입력
(따옴표 기준 화자 구분)

2

AI 자동 변환
(멀티보이스 포맷 매핑)

3

Typecast API 호출
(청크 단위 분할 요청)

4

MultiVoiceChunk 저장
(화자별 음성 파일)

5

프론트 스트리밍 재생
(배속 · 진행률 지원)

핵심 설계

화자별 자동 분기

정규식으로 대사 추출 (4종 따옴표) → 순서 기반 화자 매핑 → 800자 청크

```
// 1. 정규식 화자 분기 (4종 따옴표)
List<Segment> segs = parseMultiVoice(text);
// 2. speakerIndex별 보이스 매핑
Long voiceId = (idx == -1) ? narratorId
                  : (idx == 0) ? voice1 : voice2;
// 3. CHUNK_TEXT_SIZE = 800자
```

청크 스트리밍

긴 본문을 분할 처리 → 첫 청크 완성 즉시 재생 시작

AI 대사 자동 변환

원고 등록 시 일반 텍스트 → 멀티보이스 포맷 자동 변환

주요 기능 _ 평행우주 외전 생성

Generative Spin-off with Original Context

← 시연 메뉴

외전 생성 흐름

1

"만약에..?" 버튼 클릭
(웹소설 뷰어 하단)

2

사용자 가정 입력
(자유 텍스트)

3

원작 컨텍스트 수집
(제목 + 줄거리 + 회차 본문)

4

Gemini API 호출
(가정 + 컨텍스트 + 가이드)

5

외전 표시
(원작 분리 영역 · 500자)

핵심 설계

프롬프트 엔지니어링

제목·줄거리·회차본문·가정 4종 컨텍스트 주입 → 원작 톤 유지

```
# 4종 컨텍스트 → Gemini 2.5 Flash Lite
prompt = f"""당신은 웹소설 작가입니다.
[제목] {title} [줄거리] {summary}
[현재 회차] {content_text}
[만약에...?] {what_if}
→ 500자, 원작 톤 유지"""
```

원작 분리 표시

뷰어 하단 별도 영역에 노출 → 본편 몰입 영향 없음

능동적 콘텐츠 소비

독자가 가정을 던지고 결과를 받는 참여형 경험

주요 기능 _ 릴레이 소설

Collaborative Writing System

← 시연 메뉴

📋 릴레이 진행 흐름

1

주제 등록
(첫 작가 → RelayNovel 생성)

2

참가자 모집
(다른 작가 신청 → 순서 결정)

3

차레 알림
(Notification 자동 발송)

4

이어쓰기 제출
(RelayEntry 저장)

5

릴레이 완성
(전원 차레 종료 → 게시)

⚡ 핵심 설계



순번 기반 차레 관리

RelayEntry.entryOrder + Status enum (PUBLISHED / BLINDED) 으로 차레·상태 관리

```
@Entity
public class RelayEntry {
    private Integer entryOrder;
    private Status status;
    private String adminMessage;
    public void blind(String summary) {
        this.status = Status.BLINDED;
        this.adminMessage = summary;
    }
}
```



차레 알림 자동 발송

이전 작가 제출 완료 → 다음 작가에게 Notification 발송



관리자 모더레이션 — blind()

부적절 이어쓰기 블라인드 처리 + AI 요약본을 adminMessage 에 저장

주요 기능 _ 관리자 시스템

Operations & Moderation

← 시연 메뉴

작품 검수 흐름

1

작가 작품 등록
(상태: DRAFT)

2

검수 요청 제출
(DRAFT → PENDING)

3

관리자 검토
(콘텐츠 · AI 검수 결과 확인)

4

승인 or 반려
(APPROVED / REJECTED)

5

사용자 공개
(APPROVED 예약 → PUBLISHED)

핵심 설계

상태 머신 기반 검수

작품·릴레이·회차 공용 Status enum (7가지 상태) → 잘못된 전환 차단

```
public enum Status {  
    DRAFT,        // 임시 저장  
    PENDING,      // 검수 대기  
    APPROVED,     // 검수 완료(예약)  
    PUBLISHED,    // 공개  
    REJECTED,     // 반려  
    BLINDED,      // 관리자 블라인드  
    PRIVATE       // 비공개  
}
```

회원 정지 · 신고 처리

기간 선택 정지 · 댓글/답글 신고 일괄 처리

통합 운영 콘솔

공지 · 릴레이 주제 · TTS 보이스까지 한 화면에서 관리

SWOT 분석

Strategic Position Analysis

S Strength

내부 · 강점

- 웹툰 + 웹소설 통합 플랫폼
- AI 기능 차별화 (챗봇 · TTS · 외전 · 검수)
- 누구나 작품 등록 가능
- 릴레이 소설 협업 창작
- 체계적인 관리자/신고 시스템

O Opportunity

외부 · 기회

- 웹툰/웹소설 시장 지속 성장
- AI 콘텐츠 소비 트렌드 확산
- 신인 작가 플랫폼 수요 증가
- K-콘텐츠 글로벌 인기
- 구독 경제 정착

W Weakness

내부 · 약점

- 초기 콘텐츠 수 부족
- 브랜드 인지도 없음
- 대형 플랫폼 대비 UX 완성도 차이
- 모바일 앱 미지원
- Vision API 기반 웹툰 AI 미구현

T Threat

외부 · 위협

- 네이버/카카오 대형 플랫폼 경쟁
- 저작권 이슈
- 불법 복제 콘텐츠 위험
- 결제 시스템 보안 위험
- AI 콘텐츠 신뢰성 논란

자체 평가 _ Troubleshooting #1

결제 Race Condition

! 문제

동시에 여러 결제 요청이 들어올 때 잔액이 중복 차감될 위험이 발생할 수 있는 상황 — 트랜잭션 격리 부족

Q 원인

잔액 조회 → 차감 → 저장 사이에 다른 트랜잭션이 끼어들 수 있어 갱신 손실 (lost update) 발생

🔑 해결

낙관적 락(Wallet) + 비관적 락(Payment) — 책임 분리

```
// Wallet.java (Entity) - 낙관적 락
```

```
@Version
```

```
private Long version;
```

```
// PaymentRepository.java - 비관적 락
```

```
@Lock(LockModeType.PESSIMISTIC_WRITE)
```

```
@Query("SELECT p FROM Payment p
```

```
WHERE p.merchantUid = :uid")
```

```
Payment findByMerchantUidWithLock(...)
```

✅ 결과

동시 요청 중 하나만 성공, 나머지는 안전하게 예외 처리 — 잔액 무결성 보장

자체 평가 _ Troubleshooting #2 & #3

AI Fallback · TTS API 전환

#2 AI 따옴표 변환 — Gemini API 의존 위험

❗ 문제

Gemini API 할당량 초과 / 네트워크 오류 시 따옴표 변환 기능 전체 중단 — 작품 등록 플로우가 외부 API에 종속됨

🔍 원인

외부 API 단일 의존 구조 — Gemini 호출 실패 = 서비스 실패. 무료 할당량 한계도 운영상 리스크

🔑 해결

정규식 기반 폴백 추가 — try/except로 Gemini 실패 감지 시 `_regex_format_dialogue()` 자동 전환

✅ 결과

AI 다운타임에도 무중단 운영 — 한국어 "~다." 종결 패턴으로 대사/서술 분리, 정확도는 낮지만 서비스 연속성 확보

#3 TTS API 전환 · Google → Typecast

Google Cloud TTS 의 한계

- 한국어 보이스 종류 부족
- 감정 표현 부재 — 단조로운 억양
- 한국어 자연스러움 미흡



Typecast 전환

- 다양한 한국어 보이스 (나이/성별/감정)
- 자연스러운 한국어 억양과 감정
- 멀티보이스에 적합한 개성 있는 보이스 다수

✅ 단일 / 멀티보이스 완성도 향상 · 웹소설 청취 경험 개선

자체 평가 _ Troubleshooting #4

TTS 청크 크기 최적화

❗ 문제

한 번의 Typecast 요청에 본문 전체를 보내면 응답 지연이 길어 첫 재생까지 사용자 대기 시간이 과도하게 늘어남

🔍 원인

Typecast API 단일 요청 최대 한도(2000자)와 응답 시간이 입력 길이에 비례 — 긴 회차일수록 첫 음성 재생 지연 누적

🔑 해결

본문 청크 분할 + 순차 스트리밍 — CHUNK_TEXT_SIZE = 800자

```
// Typecast 한도(2000자) 내 균형점
private static final int CHUNK_TEXT_SIZE    = 800;
private static final int TYPECAST_MAX_CHARS = 2000;

// 청크 단위 순차 호출 → S3 → 프론트
for (String chunk : split(text, CHUNK_TEXT_SIZE)) {
    String url = typecast.generate(chunk, voiceId);
    pushToFrontend(url); // 첫 청크 즉시 재생
}
```

✅ 결과

첫 청크 완성 즉시 재생 시작 → 체감 지연 대폭 감소, 긴 본문도 무중단 스트리밍 청취

자체 평가 _ Troubleshooting #5

릴레이 소셜 동시 작성 충돌

❗ 문제

여러 유저가 동시에 "이어쓰기" 클릭 시 같은 회차에 두 명이 작성 → entryOrder 중복 저장 (예: 두 명 모두 4화 시도)

🔍 원인

lastOrder + 1 계산 → 저장까지 사이에 다른 트랜잭션 진입 가능. JPA 락만으론 분산 환경 동시성 제어 부족

🔑 해결

Redis 분산 락 + 하트비트 + 최대 점유 30분 — 좀비 락까지 방어

```
// 1차 방어: Redis 분산 락 (원자적)
redisTemplate.opsForValue().setIfAbsent(
    key, value, Duration.ofMinutes(3));

// 2차 방어: 좀비 락 + 하트비트 (최대 30분)
long elapsedMin = (now - lockedTime) / 60_000;
if (elapsedMin >= 30) redisTemplate.delete(key);
else redisTemplate.expire(key,
    Duration.ofMinutes(3));
```

✅ 결과

entryOrder 중복 차단 + 브라우저 강제 종료에도 좀비 락 자동 회수 → 무중단 협업 보장



THANK YOU

GGoBOOK · 함께 읽고, 함께 쓰는 이야기